A RANDOM WALK THROUGH RISK

Gambler's Ruin

Three betting scenarios — and the tactics that crack them

From a biased coin → closed-form formulas, clever code, a little linear algebra

3 scenarios • 1 underlying random walk

What is Gambler's Ruin?

- **The Core Model:** The **Gambler's Ruin** is a classical probability setup. It models a gambler with intial wealth k, who repeatedly plays a game until reaching a target N or going broke at 0. It represents a **random walk** between these two absorbing barriers.
- **Key Objectives:** This framework is used to answer two primary questions: the likelihood of each final outcome (success vs. ruin) and the expected duration of the game.

Some common upcoming terms

- **States:** The distinct conditions or situations the system can be in (e.g., money at an instant).
- **Transitions:** The movement of the system from one state to another, or from a state back to itself (self-loop).
- **Transition Probability:** The mathematical likelihood (between 0 and 1) that the system will move from a given current state to a specific future state.
- **Markov Chain:** A mathematical system that models a sequence of events where the probability of the next step depends solely on the **current state**, not on how the process arrived there. This defining feature is known as the **"Markov property"** or **"memorylessness"**.
- **Stationary Distribution:** A probability distribution over the states of a system that remains unchanged as the system evolves over time. (**Note:** If a system begins in this distribution, its state probabilities will remain constant indefinitely.)
- **Transition Matrix:** A square matrix where rows and columns represent states. Each element A_{ij} depicts the probability to go from state i to state j.

One coin, one number line

Every scenario is the **same picture**: a token on a number line that steps **right (prob. p)** or **left (prob. $q = 1 - p$)** each round. All that changes is the rule at the edges — and the question we ask.

Why it's solvable: the Markov property

Where you go next depends *only* on where you stand now — never on how you got there. Memoryless.

The one trick we reuse everywhere

Condition on the very first step:

$$f(\text{here}) = p\, f(\text{step right}) + q\, f(\text{step left}) \ (+\,\text{cost})$$

Three scenarios

Same random walk, three disguises. We'll state each puzzle, then crack it — watching *which* tactic the problem rewards.

1 The Classic Gambler

Fixed $1 bets. Her odds, her ruin, her time at the table.

2 The Bold Gambler

All-in betting

3 The Stock Market

A drifting price: long-run behaviour & first hits.

One idea, three costumes — and a different tool for each.

01

SCENARIO 1

The Classic Gambler

A biased coin, a target fortune, and one wall she never wants to hit.

The setup

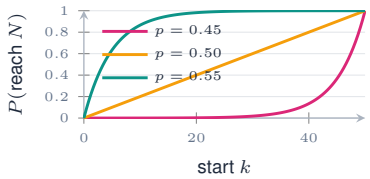
She walks in with **$k**. Every round she stakes $1 on a biased coin: heads (prob. p) and she's up a dollar; tails (prob. q) and she's down one. She stands up the instant she hits her **target $N** — or busts out at **$0**.

Three things we want to know

1. What's her chance of reaching $N before going broke?
2. If she ditches the target and just plays until ruin — can she get *infinitely* rich?
3. How long will she sit at the table before it's over?

She starts somewhere between the two walls — and the walls decide her fate.

Cracking it — her odds of winning

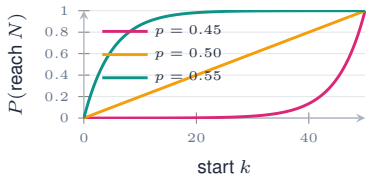


1. **Condition on the first bet.** From \$$k$ she lands on \$$k{+}1$ (prob. p) or \$$k{-}1$ (prob. q):

$$T(k) = p\,T(k{+}1) + q\,T(k{-}1).$$

It all rides on q/p. A closed form is the dream: instant, exact, no computer needed.

Cracking it — her odds of winning



1. **Condition on the first bet.** From \$$k$ she lands on \$$k{+}1$ (prob. p) or \$$k{-}1$ (prob. q):

$$T(k) = p\,T(k{+}1) + q\,T(k{-}1).$$

2. **Solve it.** Characteristic eq: $px^2 - x + q = 0$, roots 1 and q/p. General solution:

$$T(k) = A(1)^k + B(q/p)^k$$

It all rides on q/p. A closed form is the dream: instant, exact, no computer needed.

Cracking it — her odds of winning

1. **Condition on the first bet.** From \$$k$ she lands on \$$k{+}1$ (prob. p) or \$$k{-}1$ (prob. q):

$$T(k) = p\,T(k{+}1) + q\,T(k{-}1).$$

2. **Solve it.** Characteristic eq: $px^2 - x + q = 0$, roots 1 and q/p. General solution:

$$T(k) = A(1)^k + B(q/p)^k$$

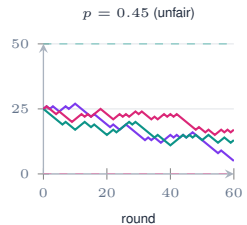
3. **Fit the ends:** Use $T(0) = 0$ and $T(N) = 1$ to solve for A and B:

$$p \neq q:\ T(k) = \frac{1-(q/p)^k}{1-(q/p)^N}; \quad p = q:\ T(k) = \frac{k}{N}.$$

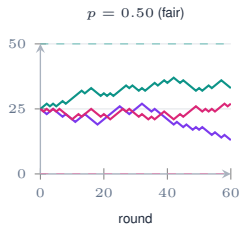
It all rides on q/p. A closed form is the dream: instant, exact, no computer needed.

3 gamblers 3 coins

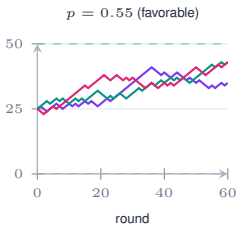
Each line is one simulated life: start at wealth 25, walls at 0 and 50. Same rules, wildly different fates.



slight disadvantage drags everyone to ruin



a coin-flip of destinies



a slight edge, most paths climb

Cracking it — forever, and for how long?

Can she get infinitely rich?

Let $N \to \infty$.

Fair or against her ($p \leq \frac{1}{2}$): ruin is **certain**, $T \to 0$.

Only with a true edge ($p > \frac{1}{2}$): $T \to 1 - (q/p)^k > 0$.

This is "risk of ruin" — why the house, on its sliver of edge, takes everyone eventually.

How long at the table?

Same trick, $+1$ for the round just played:

$$E_i = p\,E_{i+1} + q\,E_{i-1} + 1, \quad E_0 = E_N = 0.$$

Solving this gives two distinct cases: **If $\mathbf{p \neq q}$:**

$$E_i = \frac{i}{q-p} - \frac{N}{q-p}\,\frac{1-(q/p)^i}{1-(q/p)^N}$$

If $\mathbf{p = q}$ (Fair Game):

$$E_i = i\,(N-i)$$

The fair game lasts longest right in the middle.

Where this bites in real life: position sizing, insurance reserves, bankroll management.

02

SCENARIO 2

The Bold Gambler

Tired of nickel-and-dime bets, she starts going all in.

The setup

New attitude, new rule. With **less than half** the target ($k < \frac{N}{2}$) she shoves **all** her chips — double up (prob. p) or bust (prob. q). With **at least half** ($k \geq \frac{N}{2}$) she bets exactly enough to finish, $N - k$: hit the target (prob. p), or slip back to $2k - N$ (prob. q) and keep going.

Probability of winning: $P(x)$

$$P(x) = \begin{cases} 0 & x = 0 \\ pP(2x) & 0 < x < \frac{N}{2} \\ qP(2x - N) + p & \frac{N}{2} \leq x < N \\ 1 & x = N \end{cases}$$

Expected duration: $\mathbb{E}(x)$

$$\mathbb{E}(x) = \begin{cases} 0 & x = 0 \\ 1 + p\mathbb{E}(2x) & 0 < x < \frac{N}{2} \\ 1 + q\mathbb{E}(2x - N) & \frac{N}{2} \leq x < N \\ 0 & x = N \end{cases}$$

Both the odds of winning and expected time follow similar piecewise recursions.

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1 b_2 b_3 \ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1 b_2 b_3 \ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"
2. **The two moves are bit-shifts in disguise.**

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1b_2b_3\ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"

2. **The two moves are bit-shifts in disguise.**
   - Lower half ($b_1{=}0$, $x < \frac{N}{2}$): she doubles, $x \to 2x$. The fraction $\frac{2x}{N}$ is just $\frac{x}{N}$ *shifted one bit left*; the leading 0 falls off, b_2 becomes the new lead bit.

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1b_2b_3\ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"

2. **The two moves are bit-shifts in disguise.**
   - Lower half ($b_1{=}0$, $x < \frac{N}{2}$): she doubles, $x \to 2x$. The fraction $\frac{2x}{N}$ is just $\frac{x}{N}$ *shifted one bit left*; the leading 0 falls off, b_2 becomes the new lead bit.
   - Upper half ($b_1{=}1$, $x \geq \frac{N}{2}$): on a loss she drops to $2x - N$. The fraction $\frac{2x-N}{N} = 2 \cdot \frac{x}{N} - 1$ — a left shift, but now the leading 1 is the one that falls off. Either way, **one bit gets consumed per round.**

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1b_2b_3\ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"

2. **The two moves are bit-shifts in disguise.**
   - Lower half ($b_1{=}0$, $x < \frac{N}{2}$): she doubles, $x \to 2x$. The fraction $\frac{2x}{N}$ is just $\frac{x}{N}$ *shifted one bit left*; the leading 0 falls off, b_2 becomes the new lead bit.
   - Upper half ($b_1{=}1$, $x \geq \frac{N}{2}$): on a loss she drops to $2x - N$. The fraction $\frac{2x-N}{N} = 2 \cdot \frac{x}{N} - 1$ — a left shift, but now the leading 1 is the one that falls off. Either way, **one bit gets consumed per round.**

3. **So the recursion just reads the binary string.** Walk $b_1b_2b_3\ldots$ left-to-right, maintaining a running probability P. Each bit fires one of the two recurrence cases:

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1b_2b_3\ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"

2. **The two moves are bit-shifts in disguise.**
   - Lower half ($b_1{=}0$, $x < \frac{N}{2}$): she doubles, $x \to 2x$. The fraction $\frac{2x}{N}$ is just $\frac{x}{N}$ *shifted one bit left*; the leading 0 falls off, b_2 becomes the new lead bit.
   - Upper half ($b_1{=}1$, $x \geq \frac{N}{2}$): on a loss she drops to $2x - N$. The fraction $\frac{2x-N}{N} = 2 \cdot \frac{x}{N} - 1$ — a left shift, but now the leading 1 is the one that falls off. Either way, **one bit gets consumed per round.**

3. **So the recursion just reads the binary string.** Walk $b_1b_2b_3\ldots$ left-to-right, maintaining a running probability P. Each bit fires one of the two recurrence cases:
   - bit is $\mathbf{0}$: $P \leftarrow p\,P$ (matches $P(x) = p\,P(2x)$)

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1b_2b_3\ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"

2. **The two moves are bit-shifts in disguise.**
   - Lower half ($b_1{=}0$, $x < \frac{N}{2}$): she doubles, $x \to 2x$. The fraction $\frac{2x}{N}$ is just $\frac{x}{N}$ *shifted one bit left*; the leading 0 falls off, b_2 becomes the new lead bit.
   - Upper half ($b_1{=}1$, $x \geq \frac{N}{2}$): on a loss she drops to $2x - N$. The fraction $\frac{2x-N}{N} = 2 \cdot \frac{x}{N} - 1$ — a left shift, but now the leading 1 is the one that falls off. Either way, **one bit gets consumed per round.**

3. **So the recursion just reads the binary string.** Walk $b_1b_2b_3\ldots$ left-to-right, maintaining a running probability P. Each bit fires one of the two recurrence cases:
   - bit is $\mathbf{0}$: $P \leftarrow p\,P$ (matches $P(x) = p\,P(2x)$)
   - bit is $\mathbf{1}$: $P \leftarrow q\,P + p$ (matches $P(x) = q\,P(2x{-}N) + p$)

Cracking it — Probability via Binary

1. **Why binary at all?** Write the wealth fraction $\frac{x}{N}$ in binary: $0.b_1b_2b_3\ldots$ The first bit b_1 answers exactly the question the game asks first — "is x in the lower half $(b_1{=}0)$ or upper half $(b_1{=}1)$ of $[0, N]$?"

2. **The two moves are bit-shifts in disguise.**
   - Lower half ($b_1{=}0$, $x < \frac{N}{2}$): she doubles, $x \to 2x$. The fraction $\frac{2x}{N}$ is just $\frac{x}{N}$ *shifted one bit left*; the leading 0 falls off, b_2 becomes the new lead bit.
   - Upper half ($b_1{=}1$, $x \geq \frac{N}{2}$): on a loss she drops to $2x - N$. The fraction $\frac{2x-N}{N} = 2 \cdot \frac{x}{N} - 1$ — a left shift, but now the leading 1 is the one that falls off. Either way, **one bit gets consumed per round.**

3. **So the recursion just reads the binary string.** Walk $b_1b_2b_3\ldots$ left-to-right, maintaining a running probability P. Each bit fires one of the two recurrence cases:
   - bit is $\mathbf{0}$: $P \leftarrow p\,P$ (matches $P(x) = p\,P(2x)$)
   - bit is $\mathbf{1}$: $P \leftarrow q\,P + p$ (matches $P(x) = q\,P(2x{-}N) + p$)

The takeaway

What looked like a tangled piecewise recursion is actually a deterministic tour of the bits of x/N. The right *representation* of the state space turns it into an $O(\log N)$ loop — no closed form required.

Cracking it — Expected Duration via Binary

1. **Same state machine, different ledger.** The moves are unchanged: lower half doubles ($x \to 2x$, a left shift dropping a 0), upper half either ends or drops to $2x - N$ (a left shift dropping a 1). So the binary digits of $\frac{x}{N}$ *still* drive the game — only what we accumulate along the way changes.

Cracking it — Expected Duration via Binary

1. **Same state machine, different ledger.** The moves are unchanged: lower half doubles ($x \to 2x$, a left shift dropping a 0), upper half either ends or drops to $2x - N$ (a left shift dropping a 1). So the binary digits of $\frac{x}{N}$ *still* drive the game — only what we accumulate along the way changes.

2. **Why each step adds $+1$.** We're now counting *rounds*, not winning probability. Every bit consumed corresponds to one bet that actually got played, so we add 1 each time. The multiplier in front of $\mathbb{E}$ is the chance the game *survives* that round (and so still has rounds left to count).

Cracking it — Expected Duration via Binary

1. **Same state machine, different ledger.** The moves are unchanged: lower half doubles ($x \to 2x$, a left shift dropping a 0), upper half either ends or drops to $2x - N$ (a left shift dropping a 1). So the binary digits of $\frac{x}{N}$ *still* drive the game — only what we accumulate along the way changes.

2. **Why each step adds $+1$.** We're now counting *rounds*, not winning probability. Every bit consumed corresponds to one bet that actually got played, so we add 1 each time. The multiplier in front of $\mathbb{E}$ is the chance the game *survives* that round (and so still has rounds left to count).

   – bit is $\mathbf{0}$ (lower half, she doubled): $\mathbb{E} \leftarrow 1 + p\,\mathbb{E}$ (game continues only on a win, prob. p)

Cracking it — Expected Duration via Binary

1. **Same state machine, different ledger.** The moves are unchanged: lower half doubles ($x \to 2x$, a left shift dropping a 0), upper half either ends or drops to $2x - N$ (a left shift dropping a 1). So the binary digits of $\frac{x}{N}$ *still* drive the game — only what we accumulate along the way changes.

2. **Why each step adds $+1$.** We're now counting *rounds*, not winning probability. Every bit consumed corresponds to one bet that actually got played, so we add 1 each time. The multiplier in front of $\mathbb{E}$ is the chance the game *survives* that round (and so still has rounds left to count).

   - bit is $\mathbf{0}$ (lower half, she doubled): $\mathbb{E} \leftarrow 1 + p\,\mathbb{E}$ (game continues only on a win, prob. p)
   - bit is $\mathbf{1}$ (upper half, she played for the win): $\mathbb{E} \leftarrow 1 + q\,\mathbb{E}$ (game continues only on a loss, prob. q)

Cracking it — Expected Duration via Binary

1. **Same state machine, different ledger.** The moves are unchanged: lower half doubles ($x \to 2x$, a left shift dropping a 0), upper half either ends or drops to $2x - N$ (a left shift dropping a 1). So the binary digits of $\frac{x}{N}$ *still* drive the game — only what we accumulate along the way changes.

2. **Why each step adds $+1$.** We're now counting *rounds*, not winning probability. Every bit consumed corresponds to one bet that actually got played, so we add 1 each time. The multiplier in front of $\mathbb{E}$ is the chance the game *survives* that round (and so still has rounds left to count).
   - bit is $\mathbf{0}$ (lower half, she doubled): $\mathbb{E} \leftarrow 1 + p\,\mathbb{E}$ (game continues only on a win, prob. p)
   - bit is $\mathbf{1}$ (upper half, she played for the win): $\mathbb{E} \leftarrow 1 + q\,\mathbb{E}$ (game continues only on a loss, prob. q)

3. **When does it end?** The bit-walk terminates when we run out of bits — i.e. when $\frac{x}{N}$ has a finite binary expansion — exactly when the game has fully resolved.

Cracking it — Expected Duration via Binary

1. **Same state machine, different ledger.** The moves are unchanged: lower half doubles ($x \to 2x$, a left shift dropping a 0), upper half either ends or drops to $2x - N$ (a left shift dropping a 1). So the binary digits of $\frac{x}{N}$ *still* drive the game — only what we accumulate along the way changes.
2. **Why each step adds $+1$.** We're now counting *rounds*, not winning probability. Every bit consumed corresponds to one bet that actually got played, so we add 1 each time. The multiplier in front of $\mathbb{E}$ is the chance the game *survives* that round (and so still has rounds left to count).
   - bit is $\mathbf{0}$ (lower half, she doubled): $\mathbb{E} \leftarrow 1 + p\,\mathbb{E}$ (game continues only on a win, prob. p)
   - bit is $\mathbf{1}$ (upper half, she played for the win): $\mathbb{E} \leftarrow 1 + q\,\mathbb{E}$ (game continues only on a loss, prob. q)
3. **When does it end?** The bit-walk terminates when we run out of bits — i.e. when $\frac{x}{N}$ has a finite binary expansion — exactly when the game has fully resolved.

The takeaway

Two completely different quantities — a probability and an expectation — fall to the *same* bit-walk, because both inherit the same state machine. When the recursion's state space matches a familiar data representation, the representation itself is the algorithm.

03

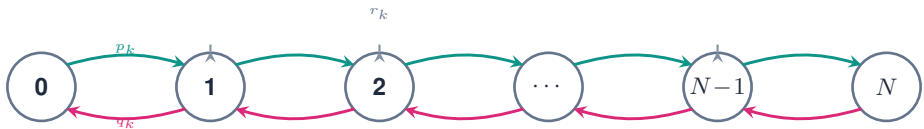
SCENARIO 3

The Stock Market

Same random walk, new costume — a drifting price.

The setup

Step away from the casino and watch a stock. Each day the price ticks **up one** (prob. p_k), **down one** (prob. q_k), or **holds** (prob. r_k) — rates that can depend on the current price k. It lives on $\{0, 1, \ldots, N\}$ and can't escape the ends.



Two questions

1. Long-run: how much time does the price spend at each level, and what's the *average* price?
2. From price a, how long until it first touches price b?

Cracking it — the long-run picture

1. Long-run proportions form a **row vector**

$$\boldsymbol{\pi} = \left[\, \pi_0,\ \pi_1,\ \pi_2,\ \ldots,\ \pi_N \,\right],$$

with $\pi_i \geq 0$ and $\sum_i \pi_i = 1$. It's the distribution that's unchanged by one more step: $\boldsymbol{\pi} = \boldsymbol{\pi} P$, where

$$P = \begin{bmatrix} r_0 & p_0 & 0 & \ldots & 0 \\ q_1 & r_1 & p_1 & \ldots & 0 \\ 0 & q_2 & r_2 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & q_N & r_N \end{bmatrix}$$

Where it spends its time — and the start point is forgotten.

Closed-form ratios, then one normalisation — a few lines of cumulative

Cracking it — the long-run picture

1. Long-run proportions form a **row vector**

$$\boldsymbol{\pi} = \big[\, \pi_0,\ \pi_1,\ \pi_2,\ \ldots,\ \pi_N \,\big],$$

with $\pi_i \geq 0$ and $\sum_i \pi_i = 1$. It's the distribution that's unchanged by one more step: $\boldsymbol{\pi} = \boldsymbol{\pi} P$, where

$$P = \begin{bmatrix} r_0 & p_0 & 0 & \ldots & 0 \\ q_1 & r_1 & p_1 & \ldots & 0 \\ 0 & q_2 & r_2 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & q_N & r_N \end{bmatrix}$$

2. **Balance neighbours:** up-flow = down-flow,

$$\pi_{i-1} p_{i-1} = \pi_i q_i \ \Rightarrow\ \pi_i = \pi_{i-1} \frac{p_{i-1}}{q_i}.$$

Where it spends its time — and the start point is forgotten.

Closed-form ratios, then one normalisation — a few lines of cumulative

Cracking it — the long-run picture

1. Long-run proportions form a **row vector**

$$\boldsymbol{\pi} = \big[\, \pi_0,\ \pi_1,\ \pi_2,\ \ldots,\ \pi_N \,\big],$$

with $\pi_i \geq 0$ and $\sum_i \pi_i = 1$. It's the distribution that's unchanged by one more step: $\boldsymbol{\pi} = \boldsymbol{\pi} P$, where

$$P = \begin{bmatrix} r_0 & p_0 & 0 & \ldots & 0 \\ q_1 & r_1 & p_1 & \ldots & 0 \\ 0 & q_2 & r_2 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & q_N & r_N \end{bmatrix}$$

2. **Balance neighbours:** up-flow = down-flow,

$$\pi_{i-1} p_{i-1} = \pi_i q_i \ \Rightarrow\ \pi_i = \pi_{i-1} \frac{p_{i-1}}{q_i}.$$

3. Chain the ratios from π_0; pin π_0 with $\sum_i \pi_i = 1$.

Where it spends its time — and the start point is forgotten.

Closed-form ratios, then one normalisation — a few lines of cumulative

Cracking it — the long-run picture

1. Long-run proportions form a **row vector**

$$\boldsymbol{\pi} = \big[\, \pi_0,\ \pi_1,\ \pi_2,\ \ldots,\ \pi_N \,\big],$$

with $\pi_i \geq 0$ and $\sum_i \pi_i = 1$. It's the distribution that's unchanged by one more step: $\boldsymbol{\pi} = \boldsymbol{\pi} P$, where

$$P = \begin{bmatrix} r_0 & p_0 & 0 & \ldots & 0 \\ q_1 & r_1 & p_1 & \ldots & 0 \\ 0 & q_2 & r_2 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \ldots & q_N & r_N \end{bmatrix}$$

2. **Balance neighbours:** up-flow = down-flow,

$$\pi_{i-1} p_{i-1} = \pi_i q_i \ \Rightarrow\ \pi_i = \pi_{i-1} \frac{p_{i-1}}{q_i}.$$

3. Chain the ratios from π_0; pin π_0 with $\sum_i \pi_i = 1$.
4. Average price $= \sum_i i\, \pi_i$.

Closed-form ratios, then one normalisation — a few lines of cumulative

Where it spends its time — and the start point is forgotten.

Cracking it — time to hit a target price

1. Let $\mathbb{E}_x$ be the expected days from x to b. Condition on the first move:

$$\mathbb{E}_x = p_x \mathbb{E}_{x+1} + r_x \mathbb{E}_x + q_x \mathbb{E}_{x-1} + 1, \qquad \mathbb{E}_b = 0.$$

*Real-world: that stationary π is **PageRank**; a first-hitting time is expected **time-to-failure**, **time-to-churn**, or how long something takes to spread.*

Cracking it — time to hit a target price

1. Let $\mathbb{E}_x$ be the expected days from x to b. Condition on the first move:

$$\mathbb{E}_x = p_x \mathbb{E}_{x+1} + r_x \mathbb{E}_x + q_x \mathbb{E}_{x-1} + 1, \qquad \mathbb{E}_b = 0.$$

2. One equation per state. Stack them into a linear system $A\mathbf{E} = \mathbf{1}$:

$$\begin{bmatrix} 1-r_0 & -p_0 & 0 & \dots & 0 \\ -q_1 & 1-r_1 & -p_1 & \dots & 0 \\ 0 & -q_2 & 1-r_2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \\ 0 & 0 & \dots & -q_N & 1-r_N \end{bmatrix} \begin{bmatrix} \mathbb{E}_0 \\ \mathbb{E}_1 \\ \mathbb{E}_2 \\ \vdots \\ \mathbb{E}_b \\ \vdots \\ \mathbb{E}_N \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

*Real-world: that stationary π is **PageRank**; a first-hitting time is expected **time-to-failure**, **time-to-churn**, or how long something takes to spread.*

Cracking it — time to hit a target price

1. Let $\mathbb{E}_x$ be the expected days from x to b. Condition on the first move:

$$\mathbb{E}_x = p_x \mathbb{E}_{x+1} + r_x \mathbb{E}_x + q_x \mathbb{E}_{x-1} + 1, \qquad \mathbb{E}_b = 0.$$

2. One equation per state. Stack them into a linear system $A\mathbf{E} = \mathbf{1}$:

$$\begin{bmatrix} 1-r_0 & -p_0 & 0 & \dots & 0 \\ -q_1 & 1-r_1 & -p_1 & \dots & 0 \\ 0 & -q_2 & 1-r_2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \\ 0 & 0 & \dots & -q_N & 1-r_N \end{bmatrix} \begin{bmatrix} \mathbb{E}_0 \\ \mathbb{E}_1 \\ \mathbb{E}_2 \\ \vdots \\ \mathbb{E}_b \\ \vdots \\ \mathbb{E}_N \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

3. Notice row b is pinned entirely: $1 \cdot \mathbb{E}_b = 0$. Since the rates vary, there's no clean formula — hand the matrix to a solver.

*Real-world: that stationary π is **PageRank**; a first-hitting time is expected **time-to-failure**, **time-to-churn**, or how long something takes to spread.*

Three ways to crack a recurrence

Notice we never used one hammer for everything. The puzzle tells you which tool is cheapest.

1. **Closed form** *(Scenario 1)* — constant coefficients turn the recurrence into a characteristic equation, and you read off a formula. Instant, exact, and it hands you genuine insight (that q/p ratio that decides everything).

2. **Spot the hidden structure** *(Scenario 2)* — the recursion's transitions ($x \to 2x$ and $x \to 2x - N$) were bit-shifts in disguise. When the states secretly match a familiar data representation, walk that representation directly. No closed form, no solver — the data layout *is* the algorithm, and we get an $O(\log N)$ loop for free.

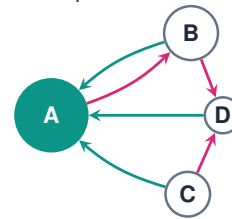
3. **Linear system** *(Scenario 3)* — once coefficients vary with the state, no clean formula survives. Stack the equations into $A\mathbf{E} = \mathbf{b}$ and hand it to a solver. Less elegant, but it always works.

Reach for the cheapest tool that works — and know when to stop hunting for a formula.

PageRank — the random surfer

Swap the number line for a **graph of web pages**. A surfer stands on a page and, each step, clicks a *random* outgoing link. Run this forever: the fraction of time spent on each page *is* its rank. Same stationary-distribution math as the drifting stock — just on a messier state space.

1. **Same balance condition.** The rank vector is fixed under one more click: $\boldsymbol{\pi} = \boldsymbol{\pi} M$, where M_{ij} is the chance of hopping $i \to j$.
2. **Vote by linking.** A page is important if important pages link *to* it — importance flows along edges, split evenly across a page's out-links.
3. **The teleport fix.** A surfer can get stuck on dead-end or cyclic pages. With prob. $1-d$ (≈ 0.15) she **teleports** to a random page. That damping makes the chain *irreducible & aperiodic* — so a *unique* $\boldsymbol{\pi}$ exists and is reached from any start.
4. **Solve by iterating.** Start uniform, apply M repeatedly — power iteration converges fast on a web-scale, billion-node graph.



A is biggest: three pages point at it. Node size $\propto$ rank.

Why it mattered

Treating links as votes — and ranking by the random walk's stationary distribution — is the idea that launched Google. The same eigenvector trick now drives citation ranking, recommendation graphs, and fraud-ring detection.

THANK YOU

One coin.
Three scenarios.
A handful of tactics.

Questions?